PYTHON · THEORY SESSION

Python Environments, Libraries & Files

pip · Conda · venv · Project Structure · Paths · Text & CSV · os, pathlib, shutil

Faculty: **Prateek Rathore** · **Ramzan Basheer** · **Lt. Col. Neeraj Sharma**
Python Programming Course

From a Single Script to a Real Project

1. **Libraries save effort** thousands of ready-made tools: NumPy, Pandas, OpenCV.
2. **Environments avoid clashes** each project keeps its own library versions.
3. **Structure keeps it tidy** data, code and results in fixed places.
4. **Files hold real data** nominal rolls, logs and results live in files, not in code.
5. **Automation** let Python create, copy, rename and back up files for you.

TODAY'S PLAN

1. Packages, modules, libraries
2. pip
3. Conda
4. Virtual environments (venv)
5. Project structure
6. Absolute & relative paths
7. Text and CSV files
8. os, pathlib, shutil

Modules, Packages and Libraries

DEFINITION

A module is a single .py file. A package is a folder of modules. A library is a collection of packages built for one purpose, such as NumPy for numbers or Pandas for tables.

KEY POINTS

1. **Standard library** comes with Python: math, os, csv, datetime.
2. **Third-party** written by others, published on PyPI.
3. **Install first** third-party libraries need pip or conda.
4. **import** loads it into your program.

PYTHON

```python
# Standard library: comes with Python
import math
from datetime import date
print(math.sqrt(144))
print(date(2026, 10, 1).strftime("%d %b %Y"))

# Third-party library: install first with pip
import numpy as np          # "np" is a short alias
print("NumPy version:", np.__version__)
```

OUTPUT

```
12.0
01 Oct 2026
NumPy version: 2.4.4
```

pip: The Python Package Installer

DEFINITION

pip is the standard tool that downloads and installs Python packages from PyPI (the Python Package Index). It is included with Python and is run from the terminal / command prompt, not inside Python.

KEY POINTS

1. **install** download and set up a package.
2. **list / show** see what is installed.
3. **freeze** record exact versions in requirements.txt.
4. **-r** rebuild the same setup on another PC.

TERMINAL / COMMAND PROMPT

```
# Install a package
pip install numpy

# Install an exact version
pip install pandas==2.2.3

# See what is installed
pip list
pip show pandas

# Save and restore a project's packages
pip freeze > requirements.txt
pip install -r requirements.txt

# Remove a package
pip uninstall numpy
```

TIP

If "pip" is not recognised on Windows, use: python -m pip install numpy

Conda: Packages and Environments Together

DEFINITION

Conda is a package manager AND environment manager that comes with Anaconda or Miniconda. It can install Python itself, Python libraries and non-Python tools, each environment kept separate.

KEY POINTS

1. **Anaconda** Conda + hundreds of data-science libraries + Jupyter.
2. **Miniconda** Conda only: small download, add what you need.
3. **-n name** every environment has its own name.
4. **Popular** widely used for data science and ML.

ANACONDA PROMPT / TERMINAL

```
# Create an environment with a chosen Python
conda create -n firing python=3.12

# Switch into it
conda activate firing

# Install libraries into this environment
conda install numpy pandas matplotlib

# List all environments
conda env list

# Leave the environment
conda deactivate
```

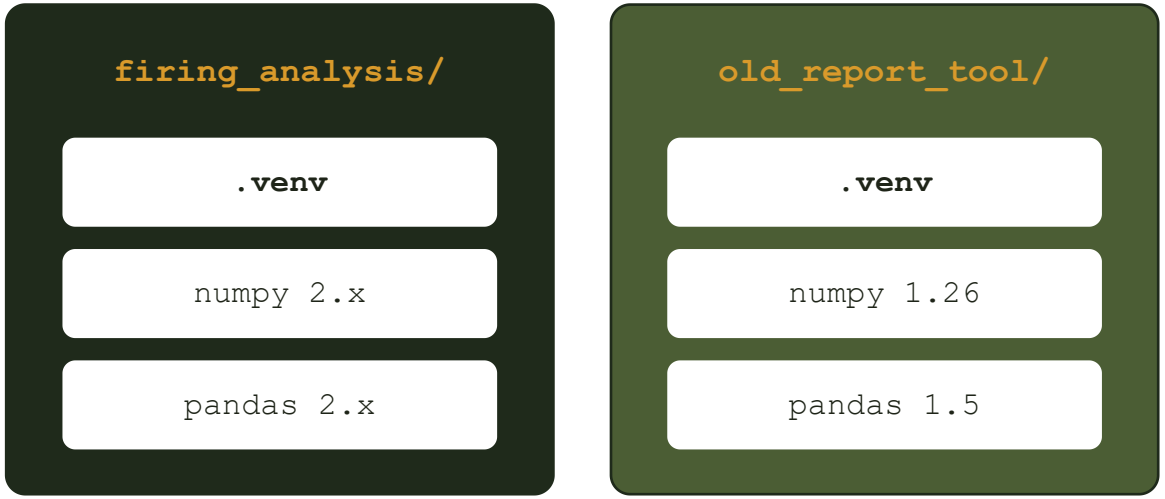
TIP

Inside an activated Conda environment you can still use pip for packages that Conda does not have.

Virtual Environments with venv

DEFINITION

A virtual environment is a separate, private folder containing its own Python and its own installed packages, created for one project. venv is the tool built into Python for creating them.



Like separate kit bags for two missions: items issued to one never mix with the other.

TERMINAL

```
# 1. Create (inside the project folder)
python -m venv .venv

# 2. Activate - Windows
.venv\Scripts\activate
# 2. Activate - Linux / macOS
source .venv/bin/activate

# 3. Install packages (only into this env)
pip install numpy pandas

# 4. Leave when finished
deactivate
```

pip vs Conda vs venv

Feature	pip	Conda	venv
What it is	Package installer	Package + environment manager	Environment creator
Comes with	Python	Anaconda / Miniconda	Python (built in)
Installs from	PyPI	Anaconda repo, conda-forge	Nothing: use pip inside it
Creates environments	No	Yes	Yes
Installs Python itself	No	Yes, any version	No: uses the installed Python
Best for	Adding libraries	Data science, complex dependencies	Light, per-project isolation

Common combinations: venv + pip (lightweight, standard Python) or Conda alone (Anaconda / Miniconda users).

A Clean Project Structure

FOLDER TREE

```
firing_analysis/
├── .venv/                  # virtual environment
├── data/                   # input files (never edit)
│   ├── marks.csv
│   └── vehicles.csv
├── src/                    # your own modules
│   ├── __init__.py
│   └── analysis.py
├── outputs/                # results, reports, charts
├── main.py                 # starting point
├── requirements.txt        # list of libraries
└── README.md               # what the project does
```

1. **One project, one folder** everything it needs lives inside.
2. **Raw data stays raw** read from data/, write to outputs/.
3. **Code in src/** functions grouped in modules.
4. **main.py** the only file you run.
5. **requirements.txt** rebuilds the environment anywhere.
6. **README.md** how to run it, for the next person.

Absolute and Relative Paths

DEFINITION

An absolute path gives the full location from the root of the drive. A relative path gives the location starting from the current working directory (the folder the program runs in).

KEY POINTS

1. **Absolute** C:\Users\cadet\firing_analysis\data (Windows)
2. **Absolute** /home/cadet/firing_analysis/data (Linux)
3. **Relative** data/marks.csv ../reports
4. **. and ..** this folder, and one folder up.

PYTHON

```python
import os
from pathlib import Path

print(os.getcwd())                    # where am I?

rel = "data/marks.csv"                # relative path
print(os.path.abspath(rel))           # full absolute path
print(os.path.exists(rel))

print(Path("..").resolve())           # one level up
```

OUTPUT

```
/home/cadet/firing_analysis
/home/cadet/firing_analysis/data/marks.csv
True
/home/cadet
```

Reading and Writing Text Files

Example: a simple duty log. Always use "with": it closes the file automatically.

PYTHON

```python
# Write (w = create new / overwrite)
with open("outputs/duty_log.txt", "w") as f:
    f.write("0600 Stand-to complete\n")
    f.write("0800 Weapon cleaning\n")

# Append (a = add at the end)
with open("outputs/duty_log.txt", "a") as f:
    f.write("1400 Range practice\n")

# Read line by line
with open("outputs/duty_log.txt", "r") as f:
    for line in f:
        print(line.strip())
```

OUTPUT

```
0600 Stand-to complete
0800 Weapon cleaning
1400 Range practice
```

FILE MODES & METHODS

- `"r"` read (default)
- `"w"` write: erases old content!
- `"a"` append at the end
- `f.read()` whole file as one string
- `f.readlines()` list of lines
- `\n` new line character

CSV Files with the csv Module

CSV (comma-separated values) is plain text that opens in Excel: one row per line, commas between columns.

PYTHON

```
import csv

rows = [["army_no", "name", "rank", "score"],
        ["1541K", "Rao", "Nk", 78],
        ["1622M", "Das", "Sep", 91],
        ["1498P", "Khan", "Hav", 64]]

with open("data/scores.csv", "w", newline="") as f:
    csv.writer(f).writerows(rows)

with open("data/scores.csv") as f:
    for rec in csv.DictReader(f):
        print(rec["rank"], rec["name"], "->", int(rec["score"]) + 5)
```

OUTPUT

```
Nk Rao -> 83
Sep Das -> 96
Hav Khan -> 69
```

REMEMBER

- `csv.writer` write rows (lists)
- `csv.reader` read rows as lists
- `csv.DictReader` read rows as dicts
- `newline=""` avoids blank lines on Windows
- `int(...)` CSV values are text

The os Module: Folders and Files

os talks to the operating system: list folders, create them, join paths, rename and delete files.

PYTHON

```
import os

print(sorted(os.listdir(".")))          # folder contents
os.makedirs("outputs/reports", exist_ok=True)

path = os.path.join("outputs", "reports", "day1.txt")
print(path)
with open(path, "w") as f:
    f.write("Range report")

new = os.path.join("outputs", "reports", "range_day1.txt")
os.rename(path, new)
print(os.listdir("outputs/reports"))
print(os.path.getsize(new), "bytes")
```

OUTPUT

```
['.venv', 'README.md', 'data', 'main.py',
'outputs', 'requirements.txt', 'src']
outputs/reports/day1.txt
['range_day1.txt']
12 bytes
```

USEFUL FUNCTIONS

- `os.getcwd()` current folder
- `os.listdir(p)` list contents
- `os.makedirs(p)` create folders
- `os.path.join()` build a path safely
- `os.path.exists()` does it exist?
- `os.remove(p)` delete a file

pathlib: The Modern Way to Handle Paths

pathlib treats a path as an object. Join paths with /, and ask the path about itself.

PYTHON

```python
from pathlib import Path

base = Path.cwd()
data = base / "data"                    # join with /
print(data.name, data.exists())

for f in sorted(data.glob("*.csv")):   # find all CSV files
    print(f.name, "|", f.stem, "|", f.suffix)

rep = base / "outputs" / "summary.txt"
rep.write_text("Total firers: 3\n")
print(rep.read_text().strip())
print(rep.relative_to(base))
```

OUTPUT

```
data True
marks.csv | marks | .csv
vehicles.csv | vehicles | .csv
Total firers: 3
outputs/summary.txt
```

PATH METHODS

- `Path("a") / "b"` join paths
- `.name` `.stem` `.suffix` parts of the name
- `.exists()` check it exists
- `.glob("*.csv")` find matching files
- `.mkdir(parents=True)` create folders
- `.read_text()` read a text file

shutil: Copy, Move, Archive and Delete

shutil ("shell utilities") handles whole files and folders: ideal for backups.

PYTHON

```
import os, shutil

shutil.copy("data/marks.csv", "outputs/marks_bak.csv")  # copy file
shutil.copytree("data", "backup/data_copy")        # copy folder
shutil.move("outputs/marks_bak.csv", "backup/")    # move file
print(sorted(os.listdir("backup")))

zip_path = shutil.make_archive("firing_backup", "zip", "backup")
print(os.path.basename(zip_path))

shutil.rmtree("backup")          # delete folder: CAREFUL!
print(os.path.exists("backup"))
```

OUTPUT

```
['data_copy', 'marks_bak.csv']
firing_backup.zip
False
```

SHUTIL FUNCTIONS

- `copy(src, dst)` copy one file
- `copytree(a, b)` copy a folder
- `move(src, dst)` move or rename
- `make_archive()` create a .zip
- `rmtree(p)` delete folder permanently

References & Further Reading

1. **Python Packaging User Guide. Install packages in a virtual environment using pip and venv.**
   https://packaging.python.org/en/latest/guides/installing-using-pip-and-virtual-environments/

2. **Python Software Foundation. venv: Creation of virtual environments.**
   https://docs.python.org/3/library/venv.html

3. **Conda Documentation. Managing environments.**
   https://docs.conda.io/projects/conda/en/latest/user-guide/tasks/manage-environments.html

4. **Python Software Foundation. File and Directory Access (os.path, pathlib, shutil) and the csv module.**
   https://docs.python.org/3/library/filesys.html

5. **Python Software Foundation. The Python Tutorial, Sec. 7.2: Reading and Writing Files.**
   https://docs.python.org/3/tutorial/inputoutput.html

6. **A. Sweigart. Automate the Boring Stuff with Python, 2nd ed. No Starch Press, 2019. Ch. 9 & 10 (Files, Organizing Files).**
   https://automatetheboringstuff.com/

Concepts follow the official documentation and texts above. All outputs shown were produced by running the code on Python 3.12. Army-context examples are original illustrations; names and figures are fictional.